

NASM - SYSCALLS

Em sistemas de 64 bits:

LEITURA E IMPRESSÃO DE CARACTER(ES):

- Sair do programa e retornar ao sistema operacional:

```
;Retorna ao sistema operacional
mov rax, 60
mov rdi, 0
syscall
```

- Leitura de caractere:

```
section .bss
    character resb 1

;Lê Caractere
mov rax, 0           ; código da syscall sys_read
mov rdi, 0           ; stdin (teclado)
mov rsi, character   ; ponteiro para a memória
mov rdx, 1           ; ler 1 byte
syscall              ; chama o sistema operacional
```

- Leitura de string:

```
section .bss
    buffer resb 50

;Leitura da String
mov rax, 0
mov rdi, 0
mov rsi, buffer
mov rdx, 50
syscall
```

OBS: Na escolha do tamanho do buffer, deve-se considerar "tamanho+1". Toda vez que uma string é lida, o caractere 0x0A sempre estará no final (Corresponde a tecla ENTER). Após a leitura da string, o registrador RAX retorna o número total de caracteres lidos sem o 0x0A.

- Impressão de caractere:

```
;Imprime Caractere
mov rax, 1           ; código da syscall sys_write
mov rdi, 1           ; stdout (tela)
mov rsi, character   ; ponteiro para o caractere
mov rdx, 1           ; escrever 1 byte
syscall              ; chama o sistema operacional
```

- Impressão de string:

```
section .data
    LF equ 10                ; Line Feed
    CR equ 13                ; Carrie return

    msg db "Hello World!", LF, CR ;String a ser impressa
    tam equ $- msg           ;Tamanho da String

;Imprime String
mov rax, 1
mov rdi, 1
mov rsi, msg
mov rdx, tam
syscall
```

TRABALHANDO COM ARQUIVOS

- Fecha o arquivo:

```
mov rax, 3                ;código da syscall sys_close
mov rdi, descriptor       ;carrega o descriptor do arquivo
rdi syscall               ;chama o sistema operacional
```

- Criar o arquivo:

```
section .data filename db "meu_arquivo.txt", 0 ; Nome do arquivo

mov rax, 85               ;Código da syscall sys_creat
mov rdi, filename         ;Ponteiro para o nome do arquivo
mov rsi, 06440            ;Permissões (Modo Octal: rw-r--r--)
syscall                   ;Chama o sistema operacional
```

OBS: RAX contém o descritor do arquivo, deve-se guardar em uma variável. Ele é necessário nas manipulações do arquivo.

- Manipula o ponteiro do arquivo:

RDX: A origem:

- 0** - A partir do início (**SEEK_SET**)
- 1** - A partir da posição atual (**SEEK_CUR**)
- 2** - A partir do final (**SEEK_END**)

RSI: Deslocamento de bytes.

Ex1: Posicionar o ponteiro no início do arquivo:

```
mov rax, 8                ; Código da syscall sys_lseek
mov rdi, descriptor       ; carrega o descriptor do arquivo
```

```

mov rsi, 0          ; Deslocamento de 0 bytes
mov rdx, 0          ; 0 = SEEK_SET (A partir do início)
syscall            ; Executa a mudança do ponteiro

```

OBS: O novo offset (em bytes a partir do início) é retornado em **RAX**

Ex2: Voltar 10 bytes a partir da posição atual:

```

mov rax, 8          ; Código da syscall sys_lseek
mov rdi, descriptor ; carrega o descriptor do arquivo
mov rsi, -10        ; Deslocamento negativo (volta 10 bytes)
mov rdx, 1          ; 1 = SEEK_CUR (A partir da posição atual)
syscall            ; Executa a mudança do ponteiro

```

OBS: O novo offset (em bytes a partir do início) é retornado em **RAX**

Descobrir o tamanho do arquivo: Se você mover o ponteiro para o fim do arquivo (**SEEK_END**) com um deslocamento de **0 bytes**, a syscall mudará o ponteiro para o final e retornará em **RAX/EAX** a posição exata em bytes. Como essa posição representa o final, ela equivale exatamente ao **tamanho total do arquivo** em bytes!

- Escrever um arquivo:

Para escrever em um arquivo, posicione o ponteiro na posição desejada e execute o seguinte trecho de programa:

```

section .data
    texto db "Olá, arquivo!", 10 ; O texto que será inserido (10 é o '\n')
    tam equ $ - texto           ; Calcula automaticamente o tamanho do texto

    mov rax, 1                  ;Código da syscall sys_write
    mov rdi, descriptor         ;carrega o descriptor do arquivo
    mov rsi, texto              ;Ponteiro para os dados
    mov rdx, tam                ;Quantidade de bytes a gravar
    syscall                    ;Escreve no arquivo

```

- Ler um arquivo:

```

section .bss buffer resb 500 ;Reserva um espaço de 500 bytes para o texto

    mov rax, 0                  ;Código da syscall sys_read
    mov rdi, rbx                ;0 descritor do arquivo de origem
    mov rsi, buffer             ;Onde salvar os dados lidos
    mov rdx, 500                ;Ler no máximo 500 bytes por vez
    syscall                    ;Executa a leitura

```

OBS: O número real de bytes que foram lidos com sucesso é retornado em **RAX**. Se retornar 0, significa que o arquivo chegou ao fim (EOF).

- Abrir um arquivo para escrita e criar caso não exista:

Para abrir um arquivo focado em **escrita** (e criá-lo automaticamente caso ele não exista), você também utilizará a syscall **sys_open**. A diferença principal está nas flags que você vai combinar usando a operação lógica OR. Em vez de usar **O_RDONLY** (Apenas Leitura), você usará uma destas opções para escrita:

- **O_WRONLY (Apenas Escrita):** Valor 1.
- **O_RDWR (Leitura e Escrita):** Valor 2.

Para adicionar a regra de criação automática, combinamos uma delas com a flag **O_CREAT** (Valor octal 01000 ou 64 em decimal).

- Se você quer **Apenas Escrita + Criar**: $1 \mid 01000 = 01010$ (ou 65 em decimal).
- Se você quer **Leitura e Escrita + Criar**: $2 \mid 01000 = 01020$ (ou 66 em decimal).

💡 **Nota importante:** Se o arquivo já existir e tiver dados, abrir apenas com essas flags vai manter o conteúdo antigo lá e posicionar o cursor no início (substituindo os bytes iniciais conforme você escreve). Se a sua intenção for **limpar o arquivo existente** para começar do zero, adicione a flag **O_TRUNC** (010000). Se o objetivo for apenas **acrescentar dados no final**, adicione a flag **O_APPEND** (020000).

Veja como configurar os registradores nas duas arquiteturas usando a combinação **Apenas Escrita + Criar (01010)**:

```
section .data
    filename db "saida.txt", 0

    mov rax, 2          ;Código da syscall sys_open
    mov rdi, filename   ;Ponteiro para o nome do arquivo
    mov rsi, 01010      ;Flags: O_WRONLY + O_CREAT
    mov rdx, 06440      ;Permissões (rw-r--r--) caso seja criado
    syscall             ;Executa
```

RAX agora contém o seu File Descriptor pronto para uso com o **sys_write**

- Abrir um arquivo para leitura e criar caso não exista:

No Linux, os valores das flags clássicas são:

O_RDONLY (Apenas Leitura): Valor 0.

O_CREAT (Criar se não existir): Valor octal 01000 (ou 64 em decimal)

```
section .data
    filename db "dados.txt", 0 ;Nome do arquivo terminado em nulo

    mov rax, 2          ; Código da syscall sys_open
    mov rdi, filename   ; Caminho do arquivo
```

```

mov rsi, 01000      ; Flags: O_RDONLY (0) | O_CREAT (01000)
mov rdx, 06660      ; Permissões padrão caso o arquivo seja criado
syscall             ; Executa a chamada do sistema

```

RAX agora contém o File Descriptor válido (positivo). Se RAX for negativo, significa que houve algum erro grave.

- Abrir um arquivo para escrita e leitura e criar caso não exista:

Para abrir um arquivo permitindo **tanto a escrita quanto a leitura** e criá-lo automaticamente caso ele não exista, você continuará usando a syscall `sys_open`.

A única mudança está no valor que você passará para o registrador de flags (RSI ou ECX). Você deve fazer uma operação lógica **OR** combinando as seguintes flags:

O_RDWR (Leitura e Escrita): Valor 2.

O_CREAT (Criar se não existir): Valor octal 01000 (ou 64 em decimal)

Combinando as duas (2 | 01000), o resultado final a ser passado é **01020** (ou 66 em decimal). Lembre-se de que, como a flag de criação está ativa, o argumento de permissões (ex: 06440) torna-se obrigatório.

```

section .data
    filename db "dados.bin", 0 ; Nome do arquivo terminado em 0

    mov rax, 2                ; Código da syscall sys_open
    mov rdi, filename         ; Ponteiro para o nome do arquivo
    mov rsi, 01020            ; Flags: O_RDWR (2) + O_CREAT (01000)
    mov rdx, 06440            ; Permissões caso o arquivo seja criado (rw-r--r--)
    syscall                   ; Executa a chamada ao sistema

```

RAX agora contém o File Descriptor (fd) pronto para escrita e leitura

Aqui está o exemplo completo em **64 bits (x86_64)**. Este código realiza todo o ciclo que conversamos:

Abre o arquivo em modo de leitura/escrita (`O_RDWR`), criando-o caso não exista (`O_CREAT`).

Escreve uma mensagem dentro dele (`sys_write`).

Move o ponteiro de volta para o início do arquivo (`sys_lseek`).

Lê o conteúdo gravado para um buffer na memória (`sys_read`).

Exibe o que foi lido diretamente na tela (`stdout` via `sys_write`).

Fecha o arquivo (`sys_close`) e encerra o programa (`sys_exit`).

```

section .data
    filename db "exemplo_64bit.txt", 0

```

```

    texto    db "Mensagem gravada e lida via Assembly!", 10
    tam_txt  equ $ - texto

section .bss
    buffer   resb 128 ;Reserva 128 bytes para ler o arquivo de volta
    fd       resq 1   ;Reserva 8 bytes (quadword) para salvar o File Descriptor

section .text
    global _start

_start:
    ; 1. ABRIR / CRIAR O ARQUIVO
    mov rax, 2          ;syscall: sys_open
    mov rdi, filename   ;caminho do arquivo
    mov rsi, 01020      ;flags: O_RDWR (2) | O_CREAT (0100o)
    mov rdx, 06440      ;permissões padrão (rw-r--r--)
    syscall

    mov [fd], rax       ;salva o File Descriptor retornado em RAX

    ; 2. ESCREVER NO ARQUIVO
    mov rax, 1          ; syscall: sys_write
    mov rdi, [fd]       ; File Descriptor do nosso arquivo
    mov rsi, texto      ; ponteiro para a mensagem
    mov rdx, tam_txt    ; tamanho da mensagem
    syscall

    ; 3. MOVER O PONTEIRO PARA O INÍCIO (SEEK_SET)
    mov rax, 8          ; syscall: sys_lseek
    mov rdi, [fd]       ; File Descriptor
    mov rsi, 0          ; mover 0 bytes
    mov rdx, 0          ; 0 = SEEK_SET (início do arquivo)
    syscall

    ; 4. LER O CONTEÚDO DO ARQUIVO
    mov rax, 0          ; syscall: sys_read
    mov rdi, [fd]       ; File Descriptor
    mov rsi, buffer     ; local onde o texto lido será guardado
    mov rdx, 128        ; ler no máximo 128 bytes
    syscall

    mov rbx, rax        ; guarda em RBX a quantidade real de bytes lidos

    ; 5. EXIBIR O CONTEÚDO LIDO NA TELA (STDOUT)
    mov rax, 1          ; syscall: sys_write
    mov rdi, 1          ; 1 = stdout (tela)
    mov rsi, buffer     ; ponteiro para o buffer preenchido
    mov rdx, rbx        ; quantidade de bytes lidos no passo anterior
    syscall

    ; 6. FECHAR O ARQUIVO
    mov rax, 3          ; syscall: sys_close
    mov rdi, [fd]       ; File Descriptor
    syscall

    ; 7. FINALIZAR O PROGRAMA (EXIT)
    mov rax, 60         ; syscall: sys_exit
    mov rdi, 0          ; código de retorno 0 (sem erros)
    syscall

```